LegalEase: AI-Powered Legal Document Generator

Project Description:

LegalEase leverages state-of-the-art Generative AI to simplify the creation of legal documents by providing customizable, accurate, and editable templates for a wide range of use cases. Users can generate employment contracts, lease agreements, NDAs, and more—all tailored to their specific inputs like involved parties, effective dates, and key terms.

Built with FastAPI and integrated with a powerful AI core, LegalEase ensures users receive professional-grade legal documents that maintain formatting standards, include custom branding such as logos and fonts, and support features like editable previews and automatic term tables. With secure handling of sensitive user data and seamless document export options (.PDF, .DOCX, .TXT), the platform bridges the gap between accessibility and legal professionalism.

LegalEase empowers entrepreneurs, professionals, and individuals to generate reliable legal content confidently—without needing a legal background.

Scenarios

Scenario 1: A startup founder enters details about a new hire and selects “Employment Contract” as the document type. LegalEase generates a comprehensive agreement with clauses for roles, responsibilities, compensation, and confidentiality. The user customizes the final wording, adds their company logo, and downloads the document as a branded PDF.

Scenario 2: A freelancer needs a Non-Disclosure Agreement for a potential client. By specifying the parties, scope of confidentiality, and effective date, they instantly receive a structured and legally sound NDA.

Scenario 3: A landlord uses LegalEase to draft a residential lease agreement. By inputting the property address, tenant details, and lease terms, the platform creates a formatted document with editable sections and a downloadable version in .DOCX format.

Technical Architecture:

Description: LegalEase utilizes a modular architecture to deliver AI-powered legal document generation. The Streamlit frontend provides a user-friendly interface for entering document details, previewing generated content, editing the output, and downloading documents. The FastAPI backend handles API requests and routes validated inputs to the AI core. The Gemini 1.5 Pro model is integrated through the GeminiDocumentGenerator to generate structured legal content. Formatting modules convert the generated content into TXT, DOCX, and PDF formats.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- FastAPI Framework Knowledge: FastAPI Documentation
- Google Gemini API Familiarity: Google Generative AI Documentation
- Streamlit for Frontend: Streamlit Documentation
- Python Programming Proficiency: Python Official Docs
- Environment Setup: pip install fastapi uvicorn streamlit python-docx fpdf Pillow requests google-generativeai python-dotenv

Project Workflow

Milestone 1: Model Selection and Architecture

- Activity 1.1: Research and select the Gemini 1.5 Pro model from Google Generative AI for legal document generation.
- Activity 1.2: Define the architecture with Streamlit frontend, FastAPI backend, Gemini AI integration, and DOCX/PDF/TXT formatting modules.
- Activity 1.3: Set up the development environment with a .env file containing the Gemini API key.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop legal document generation, editable text preview, and formatting/export as .txt, .docx, and .pdf.
- Activity 2.2: Implement the FastAPI backend with the /generate endpoint to process document_type, parties, terms, and dates.

Milestone 3: main.py Development

- Activity 3.1: Initialize the FastAPI application, configure the root endpoint, and include routes from routes.py.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the Streamlit user interface for legal document inputs.
- Activity 4.2: Create dynamic document previews, editing controls, and multi-format download options.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local deployment by configuring the virtual environment and installing dependencies.
- Activity 5.2: Launch the FastAPI backend with Uvicorn and the Streamlit frontend, then verify end-to-end functionality.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model for creating personalized and accurate legal documents using Google's Gemini model via the Google Generative AI SDK. We also define the system architecture and set up the development environment to support document generation, editing, and exporting in various formats (.docx, .pdf, .txt).

Activity 1.1: Research and Select the Appropriate Generative AI Model

1. Understand the Project Requirements

- LegalEase must generate structured legal documents such as contracts, NDAs, and agreements based on document type, parties involved, terms and conditions, and effective dates.

2. Explore Gemini Model Documentation

- Evaluate available Gemini models and review the capabilities of gemini-1.5-pro, including its long context window, structured prompting, generation speed, and compatibility with structured legal content.

3. Evaluate Model Performance

- The project documentation describes gemini-1.5-pro as capable of handling complex formatting, reasoning, and structured text generation for legal content.

4. Select the Optimal Model

- Selected: gemini-1.5-pro
- Best suited for drafting formal documents with clear sections and legal terminology.
- Capable of understanding and generating clauses based on user inputs dynamically.

Activity 1.2: Define the Architecture of the Application

5. Architectural Diagram

- Frontend: Streamlit for user interaction, document input, preview, editing, and downloads.
- Backend: FastAPI app to route input to the Gemini model and return generated content.
- AI Core: Gemini integration for content generation.
- Formatting Modules: DOCX, PDF, and TXT generators.

6. Frontend Functionality

- Users provide document type and dates through TextInput and parties and terms through TextArea.
- Buttons include Generate Document, Edit Document, and Download as TXT / DOCX / PDF.
- Styled HTML preview displays the generated legal document.

7. Backend Responsibilities

- Expose POST /generate endpoint via FastAPI.
- Accept JSON input containing document_type, parties, terms, and dates.
- Route the request to GeminiDocumentGenerator and return structured document text.

8. AI Integration Points

- GeminiDocumentGenerator forms a structured prompt and sends it to Gemini via generate_content.
- The generated text is returned to FastAPI and displayed by the frontend for editing or exporting.

Activity 1.3: Set Up the Development Environment

9. Install Python and Pip

Ensure Python 3.10+ is installed along with pip for dependency management.

10. Create a Virtual Environment
11. Install FastAPI and Uvicorn (Backend Server)

pip install fastapi uvicorn

12. Install Streamlit (Frontend UI)

pip install streamlit

13. Install Google Generative AI SDK

pip install google-generativeai

14. Install Additional Libraries

pip install python-docx fpdf Pillow requests python-dotenv

15. Set Up Application Structure

Create the project structure with the FastAPI backend, Streamlit frontend, AI core, document utilities, configuration files, and supporting modules.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

16. Legal Document Generation

Functionality: Generate legal documents such as contracts and agreements based on document type, parties, terms, and dates.

17. Document Output Formatting

Functionality: Convert raw AI-generated text into structured .docx, .pdf, and .txt files with legal formatting, logos, tables, and footers.

- format_docx(text, doc_type)
- format_pdf(text, doc_type)
- format_html_preview(text)

These use python-docx for Microsoft Word formatting, FPDF for PDF documents, and Streamlit styling for HTML preview.

18. User Interaction & Recommendations

Functionality: Accept user preferences through the Streamlit interface and reflect them in the AI-generated legal content.

19. Editable Document & Download

Functionality: Provide an editable text area and allow download in .txt, .docx, and .pdf formats.

Activity 2.2: Implement the FastAPI Backend

20. Define Routes in FastAPI

Set up the FastAPI routes required for the LegalEase service.

21. Process User Input

Use Pydantic validation together with Streamlit forms to process document type, parties, terms, and dates.

22. Integrate Gemini API Calls

Connect the backend to the GeminiDocumentGenerator so that validated user inputs are converted into generated legal content.

Milestone 3: API Logic Integration

In Milestone 3, the focus shifts to designing and integrating the backend API logic that powers document generation using the Gemini 1.5 Pro model. This milestone ensures a functional bridge between frontend inputs and the AI-powered document generation, which is the core of the LegalEase project.

Activity 3.1: Create and Configure main.py

This step involves initializing the FastAPI application, setting up the base endpoint for health-checks, and including modular API routes.

- Initialize the FastAPI app
- Define a root (/) endpoint to verify the service is running
- Include routes from the routes.py module

Activity 3.2: Handle Document Generation Logic in routes.py

This task handles user inputs from the frontend and sends them to the Gemini model for document generation.

- Define a Pydantic DocumentRequest model to validate incoming data.
- Initialize the GeminiDocumentGenerator from ai_core.gemini_generator.
- Create a POST endpoint (/generate) to accept user input and return generated legal content.

Milestone 4: Frontend Development – LegalEase

Milestone 4 focuses on creating a responsive and user-centric web interface for LegalEase using Streamlit. This milestone enables users to easily input legal details, generate AI-powered documents, preview the results, make inline edits, and download documents in multiple formats (TXT, DOCX, PDF) with rich formatting and embedded branding.

Activity 4.1: Designing and Developing the User Interface

23. Streamlit Page Configuration and Layout Setup

- Configure the Streamlit app and use a three-column layout to center-align the company logo.

24. Header and Title

- Display a subtle center-aligned title under the logo.

25. User Input Interface

- document_type: type of legal document
- parties: involved stakeholders
- terms: semicolon-separated clauses
- dates: effective date

Activity 4.2: Creating Dynamic Previews and Export Options

26. Generate Document with Backend AI

- On clicking “Generate Document”, a POST request is sent to the FastAPI backend.
- AI-generated content is sanitized and displayed dynamically.

27. HTML Preview Rendering

- The output is previewed in an elegant dark-themed scrollable card with semantic HTML.

28. Editable Document Preview

- The user can click “Edit Document” and modify the generated content using a text area.

29. Multi-Format Download Options

- .txt (Plain Text)
- .docx (Formatted Word Document with logo, headings, and table)
- .pdf (Branded PDF with logo and footer)

Core Functions & Utilities

sanitize_text(text) — Removes special characters and typographic quotes to ensure clean formatting.

format_docx(text, doc_type) — Uses python-docx to embed logo, add titles and paragraph formatting, and auto-generate a Terms table from semicolon-separated input.

format_pdf(text, doc_type) — Utilizes FPDF with a custom header/footer, centered logo, bold section headings, and bullet-style terms.

format_html_preview(text) — Converts output to stylized HTML blocks for inline display.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the LegalEase application locally using FastAPI as the backend and Streamlit as the frontend. This milestone involves setting up a virtual environment, configuring dependencies, and verifying that all modules—from the Gemini API integration to document generation and downloads—work seamlessly in a real-world local environment. This ensures the application is ready for future cloud deployment.

Activity 5.1: Preparing the Application for Local Deployment

30. Set Up a Virtual Environment

To ensure environment isolation and consistent dependency management, create and activate a virtual environment.

31. Install Dependencies

Ensure that the backend and frontend dependencies are installed from requirements.txt.

pip install -r requirements.txt

32. Configure Environment Variables

Create a .env file to securely store the Gemini API key.

Activity 5.2: Testing and Verifying Local Deployment

33. Launch the FastAPI Backend

Start the backend server using Uvicorn.

34. Launch the Streamlit Frontend

Run the app.py file to start the LegalEase UI.

35. Test All Functionalities

- Input document type, parties, terms, and date.
- Press Generate Document to get AI-generated legal content from Gemini.
- Preview in dark mode HTML.
- Edit the text.
- Download in .txt, .docx, and .pdf formats.
- Check proper layout with logo and footer.
- Check clean formatting of bullet points.
- Check correct embedding of terms in the document.

Exploring LegalEase

36. Document Type

Purpose: Type of legal document to generate.

- Agreement
- Contract
- NDA (Non-Disclosure Agreement)
- Lease Agreement
- Employment Offer Letter

Example input: Freelance Work Contract

37. Parties Involved

Purpose: Names and roles of the individuals or entities involved in the document.

- John Doe (Freelancer), ABC Corp (Client)
- Alice Smith (Tenant), XYZ Realty (Landlord)

Example input: Jane Doe (Service Provider), TechNova Inc. (Client)

38. Terms & Conditions (Use semicolons for bullet points)

Purpose: Specific clauses or rules that define the agreement. Separate each bullet point using a semicolon.

- Payment to be made within 30 days of invoice;
- The provider agrees to deliver work by the agreed deadline;
- Confidentiality must be maintained at all times;
- Either party may terminate with 15 days notice

39. Effective Date

Purpose: The date when the agreement becomes legally valid.

- April 10, 2025
- 10/04/2025
- April 15, 2025

Document Generated

Editing the Document

If the user wants to edit the document, they can do so by clicking on the “Click to Edit Document” button. A new window opens up where the document can be edited.

Downloading/ Saving the Generated Document

We can Download and save the documents in any of the following formats.

- .txt
- .docx

The document provided inculcates the use of the logo (front page) as well as the footer in the last page.

- .pdf

This format provided the user with a well formatted and ready to use legal document with Logo and Footer on all the pages.

Conclusion

LegalEase represents a transformative step in bridging the accessibility gap in legal services through the power of artificial intelligence. This project demonstrates how emerging technologies can simplify complex legal language, providing users—regardless of legal background—with a clear, concise understanding of legal documents. By integrating advanced natural language processing (NLP) models with user-centric design, LegalEase empowers individuals to navigate legal content with confidence, thereby promoting inclusivity and informed decision-making.

The platform's ability to analyze and simplify legal text, highlight key clauses, and provide plain-language summaries enhances comprehension and reduces dependency on costly legal consultations for basic understanding. This democratization of legal knowledge is critical in a world where legal literacy remains low, especially among marginalized populations. Furthermore, LegalEase supports multilingual capabilities, ensuring broader reach and utility across different regions and demographics.

From a technical standpoint, LegalEase showcases efficient integration of backend AI models, including document parsing, summarization algorithms, and user-friendly web interfaces. The project's architecture is designed to ensure scalability, data privacy, and real-time responsiveness—key elements for any modern SaaS solution. Through iterative development, testing, and feedback, the system has been refined for optimal performance and user satisfaction.

In conclusion, LegalEase is more than a technological tool—it is a catalyst for legal empowerment. By fostering transparency, accessibility, and understanding, it upholds the principle that legal information should be a public good, not a privileged resource. Future enhancements could include deeper contract analysis, integration with legal databases, and personalized recommendations. With continued innovation and support, LegalEase has the potential to revolutionize how people interact with the law, making justice more approachable for all.